# Cours 06 ter — Boucles `for`, chaînes et listes : « Méli-Mélo »

> **Avant** : cours 06 bis (conditions, boucles, `rand`) ; le `for` du cours 02 et le `vector` / `string` du cours 04. **Comment travailler** : comme au cours 06 bis, ce sont des **programmes console** — pas d'openFrameworks. Dans Visual Studio, crée un projet *Application console* et écris chaque programme dans son `main.cpp`, un programme à la fois. Lance avec `F5`.

Jusqu'ici tu as manipulé des valeurs une par une. Ce cours travaille les **séquences** : des suites de caractères (les chaînes), des suites de valeurs de n'importe quel type (les listes `vector` du cours 04, revues de près), et la boucle taillée pour les parcourir — le `for`, que tu connais du cours 02 et qu'on va pousser dans ses retranchements. En chemin, tu découvres ce qu'est un **objet**. À la fin, tout s'assemble dans un jeu de lettres : **« Méli-Mélo »** — l'ordinateur mélange un mot, tu le devines, avec un indice à la demande.

Comme au 06 bis, chaque programme est suivi d'un **Exercice** à écrire toi-même. C'est là que ça rentre.

## 1. Étape 1 — le `for` dans tous ses états : le compteur

La forme générale du `for`, vue au cours 02 :

```
for (initialisation; test; action)
    instruction;
```

L'**initialisation** prépare la boucle (souvent : créer un compteur à 0). Le **test** est évalué avant chaque tour, exactement comme dans un `while` : vrai, on fait un tour ; faux, on passe à la suite. L'**action** s'exécute à la fin de chaque tour (souvent : incrémenter le compteur). Le programme suivant compte dans tous les sens :

```
// Compteur
// Le for dans tous ses etats

#include <iostream>
using namespace std;

int main()
{
    cout << "En avant :\n";
    for (int i = 0; i < 10; i++)
    {
        cout << i << " ";
    }

    cout << "\n\nEn arriere :\n";
    for (int i = 9; i >= 0; i--)
    {
        cout << i << " ";
    }

    cout << "\n\nDe 5 en 5 :\n";
    for (int i = 0; i <= 50; i += 5)
    {
        cout << i << " ";
    }

    cout << "\n\nAvec des instructions vides :\n";
    int compteur { 0 };
    for ( ; compteur < 10; )
    {
        cout << compteur << " ";
        compteur++;
    }

    cout << "\n\nAvec deux for imbriques :\n";
    const int LIGNES { 5 };
    const int COLONNES { 3 };
    for (int i = 0; i < LIGNES; i++)
    {
        for (int j = 0; j < COLONNES; j++)
        {
            cout << i << "," << j << " ";
        }
```

```
        cout << endl;
    }

    return 0;
}
```

Cinq boucles, cinq variantes :

- **En avant** : le classique du cours 02 — `i` de 0 à 9, dix tours.
- **En arrière** : `i` part de 9, le test est `i >= 0`, l'action `i--` **décrémente**. La boucle affiche 9 8 7 … 0.
- **De 5 en 5** : l'action `i += 5` saute de cinq en cinq, et le test `i <= 50` inclut 50. Tu peux mettre ce que tu veux dans les trois cases — mais le plus courant reste : partir de 0, avancer de 1.
- **Instructions vides** : l'initialisation et l'action peuvent être… rien du tout. `compteur` est créé avant la boucle et incrémenté dans le corps — même résultat que la première boucle. Bizarre mais parfaitement légal ; ça sert quand la variable doit survivre à la boucle.
- **Imbriqués** : le `for` intérieur s'exécute **en entier à chaque tour** du `for` extérieur. Ici : pour chaque ligne `i` (5 lignes), la boucle intérieure déroule les 3 colonnes `j`, soit 15 cases affichées, ligne par ligne. Suis-le à la main : `i` vaut 0, `j` fait 0, 1, 2, retour à la ligne ; `i` passe à 1, `j` **repart de 0**… C'est LA machine à grilles — tu la retrouveras sur tous les pixels d'une image au cours 11.

Les précautions des boucles infinies (cours 06 bis) s'appliquent au `for` comme au `while` : une boucle doit pouvoir se terminer.

**Astuce** : certains programmeurs écrivent leurs boucles volontairement infinies `for (;;)` — trois cases vides, donc pas de test, donc on ne s'arrête jamais — là où d'autres écrivent `while (true)`. Les deux se valent ; dans les deux cas, c'est un `break` qui sort.

**Astuce** : tu croiseras `++i` à la place de `i++` dans beaucoup de livres. Dans la case action d'un `for`, les deux font exactement la même chose : ajouter 1.

**Essaie** : fais compter la deuxième boucle de 20 à 0 de 2 en 2, puis inverse LIGNES et COLONNES dans la dernière et compare les affichages.

**Exercice — le damier** *(nouveau programme)* :

1. deux `for` imbriqués, 8 lignes et 8 colonnes, comme la dernière boucle du compteur ;
2. dans la case `(i, j)`, affiche `#` si `(i + j) % 2 == 0`, sinon `.` — le modulo du cours 06 bis ;
3. le `endl` en fin de ligne va dans la boucle **extérieure** ;
4. lance : un damier. C'est déjà la grille des cours 12 et 15.

## 2. Les objets : des données et des fonctions dans la même boîte

Avant de manipuler les chaînes de près, un mot de vocabulaire. La plupart des choses qu'on représente dans un jeu — un vaisseau, un héros, une image — sont des **objets** : des paquets cohérents qui combinent des **données** et des **capacités**. En C++, un objet regroupe des **données membres** et des **fonctions membres**. Imagine un type `Vaisseau` défini par un programmeur : une donnée membre `energie` (un `int`) et une fonction membre `tirerLasers()`. Chaque vaisseau d'une escadrille a **ses propres valeurs** (l'un a 75 d'énergie, l'autre 10), mais tous ont la même structure.

On accède aux membres avec l'opérateur point `.` :

```
// vaisseau est un objet de type Vaisseau
if (vaisseau.energie > 10)
{
    vaisseau.tirerLasers();
}
```

`vaisseau.energie` lit la donnée membre ; `vaisseau.tirerLasers()` appelle la fonction membre. Le grand confort des objets : pas besoin de connaître leur mécanique interne pour s'en servir — comme on conduit une voiture en connaissant le volant et les pédales, pas le moteur.

Tu ne sais pas encore créer tes propres types (le `struct` du cours 07 en sera le premier pas) — mais tu utilises des objets **depuis le début** : `maListe.push_back(3)` et `maListe.size()` au cours 04, `img.load("pandaroux.jpg")` au cours 10… Chaque fois, un objet, un point, une fonction membre.

> **Essaie** : relis ton `ofApp.cpp` d'un cours précédent et repère trois appels de fonctions membres — l'objet, le point, la fonction.

## 3. Étape 2 — la chaîne comme objet : le testeur de chaînes

Le type `string` du cours 04 est justement un objet : il stocke une suite de caractères **et** fournit des fonctions membres pour la travailler. Le programme suivant en fait le tour — longueur, accès par position, recherche, effacement :

```cpp
// Testeur de chaines
// La string comme objet

#include <iostream>
#include <string>

using namespace std;

int main()
{
    string mot1 = "Game";
    string mot2("Over");
    string mot3(3, '!');

    string phrase = mot1 + " " + mot2 + mot3;
    cout << "La phrase est : " << phrase << "\n\n";

    cout << "La phrase contient " << phrase.size() << " caracteres.\n\n";

    cout << "Le caractere en position 0 est : " << phrase[0] << "\n\n";

    cout << "On change le caractere en position 0.\n";
    phrase[0] = 'L';
    cout << "La phrase est maintenant : " << phrase << "\n\n";

    for (unsigned int i = 0; i < phrase.size(); i++)
    {
        cout << "Caractere en position " << i << " : " << phrase[i] << endl;
    }

    cout << "\nLa sequence 'Over' commence en position ";
    cout << phrase.find("Over") << endl;

    if (phrase.find("aubergine") == string::npos)
    {
        cout << "'aubergine' n'est pas dans la phrase.\n\n";
    }

    phrase.erase(4, 5);
    cout << "La phrase est maintenant : " << phrase << endl;

    phrase.erase(4);
    cout << "La phrase est maintenant : " << phrase << endl;
```

```
    phrase.erase();
    cout << "La phrase est maintenant : " << phrase << endl;

    if (phrase.empty())
    {
        cout << "\nLa phrase n'existe plus.\n";
    }

    return 0;
}
```

Déroulons-le morceau par morceau.

### Trois façons de créer une `string`

```
string mot1 = "Game";
string mot2("Over");
string mot3(3, '!');
```

La première, tu la connais. La deuxième met la valeur entre parenthèses — même résultat. La troisième donne un **nombre et un caractère** : `mot3` vaut `"!!!"`, le caractère répété trois fois.

### Coller des chaînes : le `+`

```
string phrase = mot1 + " " + mot2 + mot3;
```

`phrase` vaut `"Game Over!!!"`. Le `+`, que tu connais pour les nombres, **concatène** les chaînes : on dit qu'il est **surchargé** (*overloaded*) — le même opérateur a un sens différent selon le contexte, défini par le type `string`. Rien n'explose : c'est un mécanisme normal et très courant du C++.

### La longueur : `size()`

`phrase.size()` renvoie le nombre de caractères : 12 pour `"Game Over!!!"` — les espaces comptent. C'est la même fonction membre que sur les `vector` du cours 04 : les séquences du C++ parlent la même langue.

> **Astuce** : les `string` ont aussi une fonction membre `length()`, qui renvoie exactement la même chose que `size()`. Les deux se valent.

### Lire et écrire une position : `[ ]`

`phrase[0]` est le premier caractère — `'G'`. Les positions commencent à **0**, comme dans les `vector` : le dernier caractère de nos 12 est `phrase[11]`. Et on peut **réécrire** une position : `phrase[0] = 'L';`

transforme la phrase en `"Lame Over!!!"`.

> **Piège** : ni les `string` ni les `vector` ne vérifient tes positions avec `[ ]`. `phrase[50]` sur une phrase de 12 caractères compile sans un mot… et lit ou écrit n'importe où dans la mémoire : résultats imprévisibles, plantage possible. **C'est à toi de vérifier tes indices** — on appelle ça le *bounds checking*, et ce réflexe t'accompagnera jusqu'aux pixels du cours 10 (`ofClamp` en est la version openFrameworks).

> **Astuce** : `phrase.at(50)` fait la même chose que `phrase[50]` … mais **vérifie** la position : hors bornes, le programme s'arrête net avec un message d'erreur au lieu de lire n'importe quoi en silence. Les `vector` ont le même `at()`. Pratique quand tu cherches un bug d'indice.

### Parcourir avec un `for`

```
for (unsigned int i = 0; i < phrase.size(); i++)
```

Le `for` et la `string` sont faits l'un pour l'autre : `i` visite toutes les positions valides, de 0 à `size() - 1`. Le compteur est déclaré `unsigned int` (entier **non signé**, jamais négatif) parce que `size()` renvoie ce type-là — sinon le compilateur proteste en comparant les deux.

Parcourir une séquence, c'est un geste central des jeux : mettre à jour des centaines d'unités dans un jeu de stratégie, transformer la liste des sommets d'un modèle 3D… ou visiter tous les pixels d'une image, comme tu le feras au cours 11.

### Chercher : `find()` et `string::npos`

`phrase.find("Over")` renvoie la position où la séquence commence : 5 dans `"Lame Over!!!"`. Et si on cherche quelque chose qui n'y est pas ? `find()` renvoie alors la constante spéciale `string::npos` — informellement, « une position qui ne peut pas exister ». D'où le test :

```
if (phrase.find("aubergine") == string::npos)
```

> **Astuce** : `find()` accepte un deuxième argument optionnel — la position où commencer la recherche : `phrase.find("aubergine", 5)` cherche à partir de la position 5.

### Effacer : `erase()` — et vérifier : `empty()`

Trois façons d'appeler `erase()` :

- `phrase.erase(4, 5)` : efface **5 caractères à partir de la position 4** — `" Over"` disparaît, reste `"Lame!!!"` ;
- `phrase.erase(4)` : efface **tout à partir de la position 4** — reste `"Lame"` ;
- `phrase.erase()` : efface **tout** — reste `""`, la chaîne vide.

`phrase.empty()` renvoie un `bool` : `true` si la chaîne est vide. Parfait dans un `if`.

> **Essaie** : avant de lancer, prédis l'affichage complet du programme sur papier — chaque `cout`, position par position. Puis vérifie. Change `mot3(3, '!')` en `mot3(5, '?')` et re-prédis.

> **Exercice — l'inverseur** *(nouveau programme)* :
>
> 1. lis un mot au clavier dans une `string` ;
> 2. affiche sa longueur avec `size()` ;
> 3. affiche le mot **à l'envers** : un `for` qui part de la dernière position et descend jusqu'à 0 — regarde la boucle « en arrière » du compteur, et fais attention à la position de départ (`size() - 1`, pas `size()`) ;
> 4. bonus : affiche `Palindrome !` si le mot inversé est égal au mot de départ (construis l'inverse dans une deuxième `string` avec `+`).
>
> Tu viens d'écrire ta première « lecture à l'envers » — le principe des filtres géométriques du cours 14.

## 4. Étape 3 — les listes : l'inventaire du héros

La `string` est une séquence de caractères. Le `vector` (cours 04) stocke une séquence de **n'importe quel type** : des `int` pour une table de scores, des `string` pour l'inventaire d'un héros de RPG. Au cours 04, tes listes naissaient vides et se remplissaient par `push_back` ; ici tu vas les créer **déjà pleines**, et les travailler de près. Le programme : un héros, son barda, un troc et une trouvaille.

```cpp
// L'inventaire du heros
// Les listes

#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main()
{
    vector<string> inventaire { "epee", "armure", "bouclier" };

    cout << "Ton equipement :\n";
    for (int i = 0; i < inventaire.size(); i++)
    {
        cout << inventaire[i] << endl;
    }

    cout << "\nTu echanges ton epee contre une hache de guerre.\n";
    inventaire[0] = "hache de guerre";
    cout << "Ton equipement :\n";
    for (int i = 0; i < inventaire.size(); i++)
    {
        cout << inventaire[i] << endl;
    }

    cout << "\nLe nom '" << inventaire[0] << "' contient ";
    cout << inventaire[0].size() << " caracteres.\n";

    const int NB_MAX { 4 }; // le heros ne porte pas plus de 4 objets
    cout << "\nTu trouves une potion de soin.\n";
    if (inventaire.size() < NB_MAX)
    {
        inventaire.push_back("potion de soin");
    }
    else
    {
        cout << "Tu portes trop d'objets, impossible d'en prendre un de plus.\n";
    }
    cout << "Ton equipement :\n";
    for (int i = 0; i < inventaire.size(); i++)
    {
```

```
        cout << inventaire[i] << endl;
    }

    return 0;
}
```

### Déclarer une liste déjà pleine

```
vector<string> inventaire { "epee", "armure", "bouclier" };
```

Un `vector` de `string`, comme au cours 04 — mais avec une **liste d'initialisation** entre accolades : les éléments, dans l'ordre, séparés par des virgules. La liste contient trois objets dès sa création, et `inventaire.size()` vaut 3 tout de suite. Pas de compteur à tenir à la main : c'est le `vector` qui compte. (Il faut `#include <vector>`, comme `#include <string>` pour les chaînes.)

> **Astuce** : c'est la même écriture en accolades que `int x { 400 };` depuis le cours 01, appliquée à plusieurs valeurs. Une liste vide `vector<string> inventaire;` reste possible — c'est ce que tu faisais au cours 04, quand les éléments venaient d'une boucle ou du hasard.

### Indexer une liste

```
inventaire[0] = "hache de guerre";
```

Même opérateur `[ ]` que la `string`, mêmes positions à partir de 0 : l'épée est en position 0, le bouclier en position 2. L'affichage est ton motif du cours 04 — un `for` de 0 à `size()`. Et le troc, une simple réaffectation : on **remplace** l'élément en position 0 sans changer la taille.

Les éléments restent des objets complets : `inventaire[0].size()` appelle la fonction membre `size()` de la `string` en position 0. Un point pour entrer dans la liste, un autre pour entrer dans la chaîne.

### Ajouter avec une règle

```
const int NB_MAX { 4 };
if (inventaire.size() < NB_MAX)
{
    inventaire.push_back("potion de soin");
}
```

Un `vector` grandit autant qu'on veut : le `if` n'est pas une contrainte de la machine, c'est une **règle du jeu** — un héros ne porte pas plus de quatre objets. Le `push_back` du cours 04 ajoute la potion à la fin, et `size()` passe à 4 tout seul. Son symétrique existe : `inventaire.pop_back();` **retire** le dernier élément (le héros jette son dernier objet).

**Piège** : grandir à la demande ne rend pas `[ ]` plus prudent. Une liste de 4 objets a les positions 0 à 3 — il n'y a **pas** de `inventaire[4]`, et `inventaire[10]` encore moins. Lire ou écrire là compile sans erreur et touche une zone inconnue de la mémoire : comportement imprévisible, plantage possible. Pour **ajouter**, c'est `push_back`, jamais `[ ]` au-delà de la fin. Pour **lire**, teste l'indice dès qu'il y a un doute.

**Essaie** : affiche `inventaire[10]` en fin de programme. Observe (peut-être du vide, peut-être des caractères bizarres, peut-être un plantage : c'est bien le problème). Remplace par `inventaire.at(10)` : le programme s'arrête, mais il te le **dit**. Puis remets un indice valide.

**Exercice — la liste des scores** *(nouveau programme)* :

1. déclare `vector<int> scores { 750, 1200, 400, 980, 1450 };` — la liste d'initialisation de l'étape ;
2. avec un `for`, affiche les cinq scores ;
3. trouve le **meilleur** score : une variable `meilleur` qui part du premier élément, et un `if` dans la boucle (le motif du cours 06 bis) ;
4. calcule la **moyenne** : accumule le total dans la boucle, puis divise — attention à la division entière du cours 01 : que faut-il pour une moyenne à virgule ?
5. ajoute un sixième score avec `push_back` juste après la déclaration et relance : qu'as-tu dû changer d'autre ? (Rien, si tes boucles vont jusqu'à `size()`.)

## 5. Les tableaux et les chaînes à la C

Un peu de culture, sans programme à taper. Le `vector` et la `string` sont des objets de la bibliothèque standard ; avant eux, et dans le langage C dont C++ descend, on représentait les séquences avec des **tableaux** (*arrays*) : un type, un nom, et **la taille entre crochets**, figée une fois pour toutes à la compilation.

```
int scores[5] = { 750, 1200, 400, 980, 1450 };   // cinq entiers, pas un de plus
char plateau[3][3];                               // une grille de 3 x 3 caracteres
char mot[] = "pixel";                             // une chaine a la C
```

Même opérateur `[ ]`, mêmes positions à partir de 0, même absence de vérification des bornes. Mais pas de `size()`, pas de `push_back` : la taille est dans ta tête, et c'est à toi de la trimballer dans une constante. La troisième ligne est une **chaîne à la C** (*C-style string*) : un tableau de `char` terminé par un caractère invisible, le **caractère nul** `'\0'`, qui marque la fin du texte — le compilateur le range pour toi, si bien que `mot` a 6 éléments pour 5 lettres visibles.

Tu croiseras ces tableaux dans du vieux code, dans des bibliothèques, et jusque dans openFrameworks. Les `string` cohabitent sans effort avec les chaînes à la C (on peut les concaténer, les comparer, les

passer à `find`). La morale : **sache les lire** quand ils se présentent, et **écris du `vector` et de la `string`** — les versions qui comptent pour toi et grandissent à la demande.

> **Essaie** : sur papier — combien d'éléments dans le tableau créé par `char mot[] = "pixel";` ? Que vaut `mot[5]` ? Et pourquoi `scores.size()` ne compilerait-il pas ?

## 6. Étape 4 — deux dimensions : le plateau de morpion

Une liste, c'est une ligne. Mais certaines choses d'un jeu sont **des grilles** : un échiquier, une carte, un plateau de morpion. On pourrait aplatir un échiquier dans une liste de 64 cases — c'est bien plus naturel de le déclarer en **deux dimensions**, 8 × 8 : une **liste de lignes**, où chaque ligne est elle-même une liste de cases. Le programme affiche un plateau de morpion, y joue un coup, et déclare `X` vainqueur :

```
// Plateau de morpion
// Les listes a deux dimensions

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<vector<char>> plateau { {'O', 'X', 'O'},
                                   {' ', 'X', 'X'},
                                   {'X', 'O', 'O'} };

    cout << "Voici le plateau de morpion :\n";
    for (int i = 0; i < plateau.size(); i++)
    {
        for (int j = 0; j < plateau[i].size(); j++)
        {
            cout << plateau[i][j];
        }
        cout << endl;
    }

    cout << "\n'X' joue dans la case vide.\n\n";
    plateau[1][0] = 'X';

    cout << "Le plateau est maintenant :\n";
    for (int i = 0; i < plateau.size(); i++)
    {
        for (int j = 0; j < plateau[i].size(); j++)
        {
            cout << plateau[i][j];
        }
        cout << endl;
    }

    cout << "\n'X' a gagne !\n";

    return 0;
}
```

## Déclarer et initialiser en deux dimensions

```
vector<vector<char>> plateau { {'O', 'X', 'O'},
                               {' ', 'X', 'X'},
                               {'X', 'O', 'O'} };
```

Lis le type de l'intérieur vers l'extérieur : `vector<char>` est une ligne de caractères ; `vector<vector<char>>` est une liste de ces lignes. La liste d'initialisation épouse la grille — une accolade par ligne, lisible comme le plateau lui-même : 3 lignes de 3 cases, 9 `char`.

Du coup, deux `size()` : `plateau.size()` compte les **lignes** (3), et `plateau[i].size()` compte les **cases de la ligne** `i` (3 aussi). C'est ce que lisent les deux `for`.

> **Astuce** : pour une grille vide d'une taille donnée, le `vector` accepte **un nombre et une valeur**, exactement comme `string mot3(3, '!')` à l'étape 2 : `vector<char> ligne(10, '.');` est une ligne de dix points. Et les lignes n'ont pas à être carrées : une carte de 12 lignes sur 20 colonnes est tout à fait valable.

### Indexer en deux dimensions

```
plateau[1][0] = 'X';
```

Une valeur **par dimension** : `plateau[1]` est la ligne 1 (une liste de `char`), et `plateau[1][0]` sa case 0 — celle qui contenait l'espace. Et pour afficher la grille, la paire de `for` imbriqués de l'étape 1 : `i` parcourt les lignes, `j` les colonnes, le `endl` termine chaque ligne. La double boucle et la grille, encore et toujours.

Range cette image dans un coin de ta tête : **une image est exactement ça** — une grille de cases, où chaque case contient une couleur au lieu d'un `char`. La double boucle du cours 11 parcourra les pixels comme celle-ci parcourt le morpion (et le cours 17 t'apprendra même à aplatir la grille en ruban).

> **Essaie** : fais jouer `'O'` dans une case déjà prise, relance : rien ne proteste — la liste écrase, c'est tout. Quel `if` faudrait-il pour l'interdire ?

> **Exercice — la carte du donjon** *(nouveau programme)* :
>
> 1. déclare une grille vide : `vector<vector<char>> carte;` ;
> 2. remplis-la de `'.'` avec deux `for` imbriqués : à chaque tour de la boucle extérieure (5 lignes), crée une ligne vide `vector<char> ligne;`, la boucle intérieure y fait 10 `push_back('.')`, puis `carte.push_back(ligne)` range la ligne dans la grille ;
> 3. place le héros `'@'` en ligne 2, colonne 3, et le trésor `'T'` en ligne 0, colonne 8 — après le remplissage, sinon ils seraient écrasés ;
> 4. affiche la carte avec la double boucle du morpion.
>
> Remplir, modifier, afficher : les trois gestes de toute grille. Bonus : remplace les étapes 1 et 2 par la seule ligne `vector<vector<char>> carte(5, vector<char>(10, '.'));` — cinq copies d'une ligne de

> dix points, l'Astuce appliquée deux fois.

## 7. Étape 5 — le projet : « Méli-Mélo »

Le jeu final : l'ordinateur choisit un mot au hasard, mélange ses lettres, et te l'affiche. Tu proposes des réponses jusqu'à trouver le mot. `aide` affiche un indice, `quitter` abandonne. Une partie :

```
            Bienvenue dans Meli-Melo !

Remets les lettres dans l'ordre pour trouver le mot.
Tape 'aide' pour un indice.
Tape 'quitter' pour abandonner.

Le melange est : ourssi

Ta proposition : roussi
Desole, ce n'est pas ca.

Ta proposition : aide
Elle se deplace sur l'ecran.

Ta proposition : souris

C'est ca ! Tu as trouve !

Merci d'avoir joue.
```

Tout ce que le cours a introduit y passe : une liste **2D** de `string` pour les mots et leurs indices, `rand()` pour tirer et mélanger, l'indexation `[ ]` pour échanger des lettres, et la boucle de jeu du cours 06 bis. Construisons-le en cinq morceaux.

### La liste des mots

```
// Meli-Melo
// Le jeu du mot melange, avec indice a la demande

#include <iostream>
#include <string>
#include <vector>
#include <cstdlib>
#include <ctime>

using namespace std;

int main()
{
    enum champs { MOT, AIDE, NB_CHAMPS };
    const vector<vector<string>> MOTS
    {
        {"pixel", "Le petit carre qui fait l'image."},
        {"boucle", "Elle tourne tant que c'est vrai."},
        {"clavier", "Tes dix doigts sont dessus."},
        {"souris", "Elle se deplace sur l'ecran."},
        {"melange", "C'est le nom du jeu."}
    };
```

`MOTS` est une grille de `string` comme le morpion était une grille de `char` : 5 lignes (les mots), 2 colonnes (le mot, puis son indice). Le `const` devant dit que la liste ne changera plus après sa création — on la lit, on n'y écrit pas. Nouveauté : l'**énumération** `enum champs { MOT, AIDE, NB_CHAMPS };` crée des constantes nommées qui valent 0, 1, 2 dans l'ordre. Du coup `MOTS[x][MOT]` se lit « le mot de la ligne x » et `MOTS[x][AIDE]` « son indice » — bien plus clair que `[0]` et `[1]`.

> **Astuce** : le dernier énumérateur, `NB_CHAMPS`, vaut 2 — exactement le **nombre de champs**. Terminer une énumération par un compteur est un truc classique : il sert à vérifier qu'une ligne est complète (`MOTS[x].size() == NB_CHAMPS`) ou à parcourir les champs en boucle, sans nombre magique.

### Tirer et mélanger le mot

```
srand(static_cast<unsigned int>(time(0)));

int tirage = (rand() % MOTS.size());
string motSecret = MOTS[tirage][MOT];   // le mot a trouver
string aide = MOTS[tirage][AIDE];       // son indice

string melange = motSecret;             // la version melangee
int longueur = melange.size();
for (int i = 0; i < longueur; i++)
{
    int position1 = (rand() % longueur);
    int position2 = (rand() % longueur);
    char temporaire = melange[position1];
    melange[position1] = melange[position2];
    melange[position2] = temporaire;
}
```

La graine et la recette `rand() % n` viennent du cours 06 bis — avec `MOTS.size()` comme `n` : le tirage s'adapte tout seul au nombre de mots. Le mélange : on copie le mot, puis on tire **deux positions au hasard** et on **échange** leurs caractères — via la variable `temporaire`, sinon le premier serait écrasé avant d'être sauvé. On répète autant de fois que le mot a de lettres. C'est l'indexation de la `string` en action.

### Accueillir le joueur

```
cout << "\t\t\tBienvenue dans Meli-Melo !\n\n";
cout << "Remets les lettres dans l'ordre pour trouver le mot.\n";
cout << "Tape 'aide' pour un indice.\n";
cout << "Tape 'quitter' pour abandonner.\n\n";
cout << "Le melange est : " << melange;

string proposition;
cout << "\n\nTa proposition : ";
cin >> proposition;
```

Les règles, le mélange, et une première lecture avant la boucle — le motif de la caisse enregistreuse. Au passage : aussi captivant que soit ton jeu, offre **toujours** un moyen d'en sortir.

### La boucle de jeu

```
    while ((proposition != motSecret) && (proposition != "quitter"))
    {
        if (proposition == "aide")
        {
            cout << aide;
        }
        else
        {
            cout << "Desole, ce n'est pas ca.";
        }

        cout << "\n\nTa proposition : ";
        cin >> proposition;
    }
```

On reboucle tant que la proposition n'est **ni** le mot **ni** `"quitter"` — un `&&` de deux comparaisons de `string`, avec les parenthèses de lisibilité du cours 06 bis. Dans la boucle : `aide` affiche l'indice, tout le reste est une mauvaise réponse, et on relit.

### Dire au revoir

```
    if (proposition == motSecret)
    {
        cout << "\nC'est ca ! Tu as trouve !\n";
    }

    cout << "\nMerci d'avoir joue.\n";

    return 0;
}
```

En sortant de la boucle, deux cas possibles : gagné ou abandonné. Le `if` final départage — on ne félicite que si le mot est trouvé — puis on remercie tout le monde. Assemble les cinq morceaux dans `main.cpp`, lance, et joue.

> **Essaie** : ajoute trois mots de ton cru à la liste (mots et indices **sans accents** — la console ne les affiche pas bien). Que faut-il changer d'autre pour qu'ils soient tirés ? Rien : `MOTS.size()` suit. Avec un tableau à la C de la section 5, il aurait fallu corriger une taille à la main — et l'oublier aurait tiré hors de la liste.

> **Exercice — le mélange honnête.** En suite du jeu, deux chantiers :

1. compte les propositions dans une variable `essais` (l'`aide` ne compte pas — un `if` … ou un `continue` ?) et annonce le total dans le message de victoire ;
2. il y a un bug caché : le mélange peut retomber **sur le mot lui-même** — partie gagnée d'avance. Enferme le bloc de mélange dans un `do … while` (cours 06 bis) qui remélange tant que `melange == motSecret`.

## Questions et réponses

**`while` ou `for`, lequel est le meilleur ?** Aucun des deux dans l'absolu : prends celui qui colle au besoin. Les cas qui crient « `for` » : compter, et parcourir une séquence.

**`break` et `continue` marchent-ils dans un `for` ?** Oui, exactement comme dans un `while` : `break` sort de la boucle, `continue` saute au tour suivant.

**Pourquoi les compteurs s'appellent-ils toujours `i`, `j`, `k` ?** Pure tradition — elle remonte aux premières versions du FORTRAN, où les variables entières devaient commencer par certaines lettres, dont `i`, `j` et `k`.

**Pourquoi `#include <string>` alors que `int` ou `char` n'ont besoin de rien ?** `int` et `char` sont des types **intégrés** au langage, toujours disponibles. `string` est défini dans la bibliothèque standard, dans le fichier `string` — comme `rand()` vit dans `cstdlib`.

**D'où vient le nom « chaînes à la C » ?** Du langage C, où les chaînes sont des tableaux de caractères terminés par `'\0'`. Quand le type `string` est arrivé en C++, il a fallu un nom pour distinguer l'ancienne méthode.

**Pourquoi préférer les objets `string` et `vector` aux tableaux à la C ?** Surtout parce qu'ils sont **redimensionnables** et qu'ils **se comptent eux-mêmes** : pas de limite de taille à prévoir à l'avance, pas de constante à trimballer, `size()` dit toujours vrai. Réserve les tableaux et chaînes à la C aux codes existants qui les utilisent.

**C'est quoi, la surcharge d'opérateur (*operator overloading*) ?** Donner à un opérateur familier un sens supplémentaire dans un nouveau contexte : `4 + 6` additionne, `mot1 + mot2` concatène. Le contexte lève l'ambiguïté. Et oui, `+=` est surchargé aussi : `phrase += "!"` colle à la fin.

**`size()` ou `length()` ?** Les deux renvoient la même chose. Au choix.

**C'est quoi, une fonction prédicat ?** Une fonction qui renvoie `true` ou `false`. `empty()` en est une.

**Que se passe-t-il si j'écris hors des bornes d'une liste avec `[ ]` ?** C++ te laisse faire… et tu modifies une zone inconnue de la mémoire : résultats imprévisibles, plantage possible. La vérification des bornes, c'est ton travail — ou celui de `at()`, qui arrête le programme proprement.

**Pourquoi `[ ]` ne vérifie-t-il pas, puisque `at()` sait le faire ?** Pour la vitesse : vérifier coûte un test à chaque accès, et un programme qui parcourt des millions de pixels par seconde ne veut pas le payer quand l'indice est sûr. `[ ]` te fait confiance ; `at()` te protège. À toi de choisir selon le contexte.

**À quoi servent les listes de listes, si on peut tout aplatir ?** À rester intuitif : un échiquier se pense en `echiquier[8][8]`, pas en `echiquier[64]`. (Le cours 17 te montrera quand même l'aplatissement — les

images sont stockées comme ça.)

## Questions de réflexion

1. Dans ton jeu préféré, qu'est-ce qui ferait de bons objets ? Avec quelles données membres et quelles fonctions membres ?
2. Quels avantages une liste a-t-elle sur une poignée de variables séparées ( `score1` , `score2` , `score3` …) ?
3. Quelles limites impose la taille fixe d'un tableau à la C ? Comment le `vector` les lève-t-il ?
4. Avantages et inconvénients de la surcharge d'opérateurs ?
5. Quels jeux pourrais-tu construire avec des `string` , des `vector` et des `for` comme outils principaux ?

## Exercices

1. **Lire avant de lancer.** Qu'est-ce qui cloche dans cette boucle ? (Pense aux positions valides d'une `string` .)

```
for (unsigned int i = 0; i <= phrase.size(); i++)
{
    cout << "Caractere en position " << i << " : " << phrase[i] << endl;
}
```

2. **Lire avant de lancer.** Et dans ce programme ? (Combien d'éléments la liste contient-elle au moment du premier `=` ?)

```
vector<int> scores;
scores[0] = 750;
scores[1] = 1200;
cout << scores.size() << endl;
```

3. **Le score de Méli-Mélo** *(plus costaud)*. Ajoute un système de points au jeu : le mot rapporte d'abord sa longueur en points ( `size()` ), chaque mauvaise proposition en retire un, et demander l'aide divise le capital restant par deux. Affiche le score à la victoire — et pas de score négatif : borne-le à zéro avec un `if` . À toi de décider où chaque règle s'insère dans le programme.

## Ce qu'il faut retenir

- Le `for` a trois cases — initialisation, test, action — toutes libres : compter en avant, en arrière, par pas, ou laisser des cases vides. Le test se comporte comme celui d'un `while`.
- Deux `for` **imbriqués** parcourent une grille : la boucle intérieure s'exécute en entier à chaque tour de l'extérieure. C'est la machine des cours 11, 12 et 15.
- Un **objet** combine données membres et fonctions membres, accessibles par le point : `phrase.size()`, `img.load(...)`. Tu en utilises depuis le cours 04.
- La `string` est un objet-séquence : `size()` / `length()`, `[ ]` pour lire et écrire une position (à partir de 0), `find()` et `string::npos`, `erase()`, `empty()`. Le `+` concatène — un opérateur **surchargé**.
- Un `vector<type>` stocke une séquence de n'importe quel type et grandit à la demande : **liste d'initialisation** entre accolades `vector<int> scores { 750, 1200 };`, indexation `[ ]`, `size()`, `push_back` / `pop_back`, et `at()` qui vérifie la position.
- **Ni les `string` ni les `vector` ne vérifient les bornes avec `[ ]`** : positions valides de 0 à taille – 1, et c'est à toi d'y veiller. Pour ajouter, `push_back` — jamais `[ ]` au-delà de la fin.
- Les **tableaux à la C** (`int scores[5]`) ont une taille figée et ne se comptent pas ; les **chaînes à la C** sont des tableaux de `char` terminés par `'\0'`. Sache les lire, préfère `vector` et `string`.
- Une liste de listes — `vector<vector<char>> plateau` — représente une grille ; on indexe avec une valeur par dimension, `plateau[i][j]`, et `plateau.size()` compte les lignes. Une image est une grille comme celle-là.
- Une **énumération** (`enum`) nomme des constantes qui se suivent (0, 1, 2…) ; un dernier énumérateur compte les éléments.

---

*Ce cours est l'adaptation française du chapitre 3 de « Beginning C++ Through Game Programming » de Michael Dawson (Cengage Learning) : « For Loops, Strings, and Arrays: Word Jumble ».*